

ZOHO PAYMENT RECONCILIATION & COLLECTIONS INTELLIGENCE PLATFORM

Comprehensive Technical Architecture, System Engineering, Business Value, and Operational Manual

Document Version: 1.0.0
Target System: Zoho CRM & Zoho Books Integrated Web Application
Backend Stack: Node.js (v18+) & Express Enterprise Middleware
Frontend Stack: Vanilla HTML5 / ES6 JavaScript / Modern CSS (Zero Build Requirement)
Security Framework: Server-Side OAuth2 Refresh Token Authorization
Publication Date: September 2026
Scope: End-to-End Application Architecture, Core Engines, Business Use Cases & Operational Playbook

EXECUTIVE SUMMARY & SYSTEM PURPOSE:
*The **Zoho Payment Reconciliation & Collections Platform** is a high-performance, enterprise-grade web application designed to bridge the gap between sales agreements recorded in **Zoho CRM** ("Closed Won" deals) and financial receipts logged in **Zoho Books**.*

The platform eliminates revenue leakage, automates multi-tier customer accounting cross-checks, provides predictive monthly collections forecasting across instalment schedules, and equips credit control teams with real-time, interactive chase tools and native dual-sheet Excel exports.

DOCUMENT STRUCTURE

Section	Title	Core Scope & Content
1	How the Application Works	Functional workflows, dual processing engines, 3-tier customer matching, insight dashboards, keep-alive heartbeat streaming, session persistence.
2	How the Application is Made	Full technical breakdown, Node.js/Express architecture, lib/ module specifications, OAuth2 security, frontend shell, native XLSX OOXML builder.
3	Uses of the Application	Business value propositions, revenue leakage prevention, cash flow forecasting, sales owner accountability, AR ageing & risk assessment.
4	Daily Operational Guide	Day-to-day SOPs for Finance, Billing, Credit Control, Sales Managers, and IT Administrators; step-by-step usage procedures and troubleshooting.

1. How the Application Works

1.1 System Architecture & Business Problem

In enterprise operations, sales teams manage deals within **Zoho CRM**, setting structured payment plans (e.g., Registration Fee, Instalments 1-3, Educational/Commercial Loans, or Full Upfront Payments). Meanwhile, finance teams issue invoices and record receipts in **Zoho Books**. Historically, cross-verifying whether deals marked as 'Closed Won' in CRM actually resulted in realized invoice payments in Books was a manual, error-prone task. Furthermore, evaluating upcoming cash flows across instalment schedules required cumbersome manual spreadsheets.

This web application operates as a real-time intelligence layer over Zoho CRM and Zoho Books. It provides two core computational engines:

- **1. Reconciliation Engine (/):** Performs deal-by-deal audit matching between CRM deal payment status and actual Zoho Books invoice payments.
- **2. Collections & Analytics Engine (/collections.html & 6 Insight Views):** Explodes Closed Won deals into granular scheduled payment components and projects expected vs. collected revenue month-by-month.

1.2 The Reconciliation Engine & 3-Tier Customer Matching

When an administrator selects a deal closing date range (e.g., Jan 1, 2026 to Mar 31, 2026) and clicks **Run Report**, the reconciliation pipeline executes as follows:

1. **CRM Deal Fetching:** The server queries Zoho CRM v2 API for all deals in Stage = 'Closed Won', handling API pagination up to 10,000 records.
2. **Date Range Scoping:** Filters deals based on their exact Closing_Date.
3. **CRM Expected Total Calculation:** Analyzes the deal's Payment_Type (Full Payment, Instalments, or NACL Loan) and sums all components marked as paid via CRM checkboxes (Registration_Amount_Paid, Instalment_1_Amount_Paid, etc.).
4. **3-Tier Books Customer Matching:** For each deal, the system connects to Zoho Books to discover the matching customer contact and retrieve their invoices using a strictly ordered 3-tier algorithm:

Tier	Search Mechanism	Matching Criteria & Logic
Tier 1	Direct Email Search	Searches Books contacts by CRM Email. If matching contact is found with >0 invoices, customer ID is bound.
Tier 2	Verified Phone Search	Extracts last 10 digits of CRM Phone. Searches Books contacts by phone. Only matches if the candidate's last 10 digits match CRM phone exactly (prevents loose substring mismatches).
Tier 3	Contact Person Fallback	Executes search_text query using email or phone last 10 digits to catch secondary contacts listed under corporate customer records.

5. **Financial Reconciliation & Status Categorization:** Compares the total paid amount calculated from CRM against total invoice payments recorded in Books (excluding credit notes). Assigns one of four validation results:

Result	Condition	Operational Meaning
MATCH	CRM Paid - Books Paid <= Rs.1	CRM payment records agree perfectly with Zoho Books invoices.
MISMATCH	CRM Paid != Books Paid	Discrepancy detected (e.g. CRM shows paid but Books invoice is pending, or vice versa).
MISMATCH	Customer Not Found	No matching customer contact exists in Zoho Books across all 3 tiers.
PENDING	CRM Paid = 0 & Books Paid = 0	Deal has no payments marked paid in CRM and no receipts in Books.

1.3 Collections & Month-Wise Bucketing Engine

The Collections Engine (/collections.html) shifts focus from accounting audits to cash flow forecasting and credit management. It explodes every Closed Won deal into its constituent scheduled payments (Instalments 1-3, Loan, Full Payment) and buckets them into calendar months based strictly on their CRM due dates.

- **Due Date Parsing:** Enforces strict ISO calendar validation (yyyy-MM-dd). Invalid dates (e.g. 2026-02-29 or 10/03/2026) are flagged in audit logs rather than guessed, preventing money from misallocating into wrong months.
- **Full Payment Handling:** Full payments carry no due date by definition. They are scoped to the date range using the deal's Closing_Date and categorized in a dedicated 'No due date' bucket.
- **Paid vs. Pending Determination:** A payment component is marked PENDING only if its CRM paid checkbox is false. If the expected date has passed today's date, it is classified as **Overdue / Past Due**.

1.4 The 7 Specialized Insight & Performance Dashboards

To serve different operational stakeholders, the application provides specialized analytical dashboards built on top of the collections dataset:

Dashboard Page	Target Audience	Core Question Answered & Capabilities
Collections (/collections.html)	Finance & Leadership	Monthly expected vs. collected revenue bar chart, KPI cards, pending drill-down modal, dual-sheet Excel chase list export.
By Deal Owner (/owner-collections.html)	Sales Directors	Collection performance breakdown by sales representative and their underlying customer accounts.
Batch Performance (/batches.html)	Course Operations	Evaluates collection rates and outstanding shortfalls grouped by student intake batches.
Ageing & Overdue (/ageing.html)	Credit Control	Categorizes unpaid receivables into ageing buckets (0-30, 31-60, 61-90, 90+ days past due) with contact details.
Payment Mix (/mix.html)	Strategy & Product	Compares collection velocity across payment shapes (Full Payment vs. Instalments vs. Educational Loans).
Collection Trend (/trend.html)	Executive Team	Cumulative shortfall growth vs. monthly collection rates over time.
At-Risk Deals (/risk.html)	Collection Agents	Prioritizes high-exposure overdue accounts with phone/email and expanding payment schedule details.

1.5 Real-Time HTTP Keep-Alive Heartbeat Streaming

A full reconciliation report across thousands of deals requires making thousands of HTTP calls to Zoho Books. On cloud deployment platforms (such as Render, Heroku, or AWS ALB), edge proxies automatically terminate any connection that remains idle for ~100 seconds, resulting in browser errors like Unexpected end of JSON input.

To solve this cleanly without complex WebSockets, the application implements a custom HTTP streaming heartbeat protocol (lib/heartbeat.js):

1. Immediately flushes HTTP 200 headers with Content-Type: application/json and X-Accel-Buffering: no.
2. Dribbles a single space character (' ') every 10 seconds while backend API loops execute.
3. Since leading whitespace is ignored by standard JSON parsers, when the final payload arrives, JSON.parse() processes the complete result seamlessly without proxy disconnections.

2. How the Application is Made

2.1 Architectural Overview & File Hierarchy

The project follows a clean, decoupled modular architecture. The backend is built with **Node.js & Express**, while the frontend consists of high-performance **Vanilla HTML5/JS/CSS** requiring no webpack, babel, or node_modules client dependencies.

Path / Module	Component Type	Description & Engineering Responsibility
server.js	Server Entry	Express application initialization, trust proxy config, JSON error interceptor, static asset middleware, global error handling.
lib/zohoAuth.js	OAuth Module	Manages OAuth2 token refresh flow, in-memory caching (~60s refresh buffer), and diagnostic error messages.
lib/zohoClient.js	API Transport	Wraps fetch with OAuth headers, handles HTTP 204 No Content safely, pages through CRM deals, executes 3-tier customer search.
lib/payments.js	Domain Logic	Single source of truth for payment schedule breakdown, paidFlags extraction, strict ISO date parsing, lookup display formatting.
lib/reconcile.js	Domain Logic	Calculates CRM expected total, matches Books invoices, computes overdue days per instalment, assigns MATCH / MISMATCH / PENDING.
lib/collections.js	Domain Logic	Month-wise bucketing engine, aggregate metrics, exception handling (missing dates, invalid dates, unrecognised types).
lib/heartbeat.js	Middleware	HTTP streaming keep-alive protocol (dribbles ' ' every 10s) to prevent cloud proxy idle timeouts.
lib/staticAssets.js	Middleware	Cache-busting middleware: stamps HTML script/CSS URLs with ?v=BUILD_ID and enforces no-store headers.
routes/report.js	Express Controller	POST /api/run-report endpoint: executes reconciliation with concurrency control (CONCURRENCY = 5).
routes/collections.js	Express Controller	POST /api/collections endpoint: computes month-wise collection forecast payload.
public/shell.js	Frontend Framework	Unified dashboard shell: collapsible left nav rail, global date/filter state, KPI row builder, Shell.openDetail() panel.
public/viz.js	Frontend Toolkit	Viz.rupee() currency formatter, SVG bar/line chart engines (Viz.bars, Viz.lines), sortable Viz.DataTable, 10-min session cache.
public/xlsx.js	Excel Builder	In-house OOXML zip writer generating native binary .xlsx files (Pending Detail chase list & By Deal summary) in pure JS.

2.2 Deep-Dive into Backend Modules (lib/)

• lib/zohoAuth.js -- OAuth2 Token Manager

Manages OAuth2 authentication with Zoho API services. Instead of using short-lived browser sessions, it reads a long-lived Zoho_REFRESH_TOKEN from environment variables and exchanges it for a 3600-second access token. Access tokens are cached in memory and automatically refreshed 60 seconds before expiration. If token refresh fails, it provides detailed diagnostic messages pinpointing bad client IDs or expired grant codes.

• lib/zohoClient.js -- API Transport & Pagination Engine

Wraps native fetch with Zoho authorization headers. Crucially, it handles Zoho's HTTP 204 'No Content' responses gracefully by returning an empty object rather than triggering JSON parse crashes. It includes fetchAllClosedWonDeals(), which pages through Zoho CRM search API (200 records/page, up to 50 pages) inspecting info.more_records. It also implements findBooksCustomerAndInvoices() for 3-tier customer resolution.

• **lib/payments.js -- Single Source of Truth for Payments**

Acts as the single source of truth for payment schedule extraction. Defines paidFlags(deal) to normalize boolean checkboxes, lookupName(raw) to safely parse Zoho lookup objects, and normaliseDate(raw) for strict ISO date validation. The function scheduledPayments(deal) breaks down a deal into its active payment components, capturing deal name, owner, contact email, phone, batch name, closing date, and due date.

• **lib/reconcile.js -- Reconciliation Logic**

Implements reconcileDeal(deal, orgId). Calculates expected CRM paid total based on deal payment type, invokes Books customer matching, sorts valid invoice dates chronologically, computes overdue days per instalment using date arithmetic, and classifies the deal into MATCH, MISMATCH, or PENDING.

• **lib/collections.js -- Collections Calculation Engine**

Implements buildCollections(deals, fromDate, toDate, today). Generates continuous month keys between start and end dates, iterates over all scheduled payments, aggregates paid/pending/overdue amounts per month, and compiles exception metadata (missing due dates, invalid dates, unrecognised payment types).

2.3 Deep-Dive into Frontend Framework (public/)

• **public/viz.js -- Data Visualization & Component Toolkit**

A lightweight, zero-dependency utility library. Provides Viz.rupee(amount) for Indian currency formatting (e.g. Rs. 1,50,000), Viz.bars and Viz.lines for rendering responsive inline SVG charts using CSS custom properties, Viz.DataTable for rendering interactive sortable tables with live search, and a 10-minute client session cache.

• **public/shell.js -- Dashboard Application Shell**

Provides the shared layout infrastructure for all analytical pages. Features a hover-expanding left navigation rail, unified top date range controls, global filter chips (batch, owner, payment type, search query), KPI summary cards, and the central Shell.openDetail(deal) slide-over panel for displaying complete customer payment schedules safely via textContent DOM injection.

• **public/xlsx.js -- Native Binary OOXML Excel Writer**

An in-house XML zip builder that generates native Microsoft Excel (.xlsx) binary workbooks in pure JavaScript without third-party dependencies. It writes native numeric types, currency formats, and Excel date serials across multiple worksheets (e.g., 'Pending Detail' chase list and 'By Deal' summary), allowing finance teams to filter, sort, and sum data directly in Excel.

• **lib/staticAssets.js -- Cache Invalidation Middleware**

Prevents browsers from executing stale client scripts after server deployments. Replaces script and stylesheet tags in HTML files with ?v=BUILD_ID query parameters (tied to git commit hash) and enforces HTTP Cache-Control: no-store headers on HTML documents.

3. Uses of the Application

3.1 Primary Business Value Propositions

The application serves as a critical operational bridge between Sales, Finance, and Executive Management. Below are the primary business use cases and financial impacts delivered by the platform:

Business Goal	Problem Addressed	Platform Feature & Solution
---------------	-------------------	-----------------------------

Revenue Leakage Prevention	Deals marked 'Closed Won' in CRM where sales reps checked paid boxes, but no actual invoice or payment exists in Books.	Reconciliation Engine (/) flags MISMATCH deals instantly, highlighting exact dollar discrepancies and missing Books customer contacts.
Cash Flow Forecasting	Uncertainty regarding upcoming instalment receipts and expected monthly cash inflows.	Collections Dashboard (/collections.html) aggregates instalment schedules by month, showing exact expected, collected, and pending totals.
Sales & Owner Accountability	Lack of visibility into which deal owners are carrying heavy overdue receivables.	By Deal Owner Dashboard (/owner-collections.html) breaks down collected vs. outstanding funds per sales representative and account.
Credit Control & Collections	Collection agents wasting time calling paid accounts or manually building chase lists.	At-Risk Deals (/risk.html) & Ageing Dashboard (/ageing.html) prioritize worst-overdue accounts with direct email/phone details and instant Excel exports.
CRM Data Quality Auditing	Garbage data in CRM (invalid due date formats like '10/03/2026', unmapped payment types).	Status Line Warnings surface unparseable dates and incomplete records, enabling CRM admins to fix data hygiene issues proactively.

3.2 Impact Summary across Key Roles

- **CFO & Finance Directors:** Gain total visibility over realized revenue vs. un-invoiced commitments, ensuring strict financial compliance.
- **Accounts Receivable & Billing Specialists:** Save 15+ hours weekly by replacing manual cross-checking between CRM and Books with 1-click automated matching.
- **Collection & Credit Control Teams:** Download pre-formatted, sorted Excel chase lists with phone numbers, emails, and overdue days pre-calculated.
- **Sales Operations & Managers:** Monitor individual rep collection efficiency during pipeline reviews and enforce payment collection before closing deals.

4. Daily Operational Guide (Day-to-Day Use)

4.1 Step-by-Step Daily SOP for Finance & Accounts Teams

Morning Audit Routine (09:00 AM) -- Running the Reconciliation Report:

1. Open the application URL and navigate to the **Reconciliation** page (/).
2. Select the desired deal closing date range (e.g. Current Month or Previous Quarter) using the date pickers.
3. Click **Run Report**. The system displays a live heartbeat status while fetching data from Zoho CRM and Books.
4. Once loaded, filter the table by status: click MISMATCH to inspect discrepancies.
5. Review the **Remarks** column for each mismatch deal:
 - If *'Customer not found in Books'*: Verify customer email/phone in CRM and create the contact in Books.
 - If *'CRM paid > Books paid'*: Check if payment was collected but invoice was not updated in Books.
 - If *'Books paid > CRM paid'*: Update CRM paid checkboxes to reflect realized payments.
6. Click **Export to CSV** to save the full audit log for accounting records.

4.2 Step-by-Step Daily SOP for Collection Agents & Credit Control

Daily Collection & Follow-Up Routine:

- 1. Navigate to **Collections** (/collections.html) or **At-Risk Deals** (/risk.html).
- 2. Click on the **Pending Tile** or click any month bar in the collection chart containing pending money.
- 3. The **Pending Drill-Down Modal** opens, listing all unpaid instalment payments sorted by most-overdue first.
- 4. Click **Download Excel**. This downloads a native .xlsx file containing:
 - Sheet 1: *Pending Detail* (Chase list with customer names, emails, phone numbers, overdue days, and amounts).
 - Sheet 2: *By Deal* (Deal-level aggregate totals and earliest due date).
- 5. Use the contact details in the modal or Excel sheet to make collection calls or send payment reminders.
- 6. Click any customer row in the table to open Shell.openDetail() and inspect their complete payment history.

4.3 Weekly Review SOP for Sales Management

- 1. Open **By Deal Owner** (/owner-collections.html).
- 2. Review the top chart to evaluate outstanding balances across account managers.
- 3. Filter by a specific deal owner to see their customer portfolio in the bottom chart and table.
- 4. Address high-pending accounts during weekly 1-on-1 sales pipeline meetings.

4.4 Troubleshooting & System Maintenance

Symptom / Error	Root Cause	Resolution Procedure
invalid_code on token refresh	ZOHO_REFRESH_TOKEN in .env is invalid or an unexchanged grant code was pasted.	Generate a fresh grant code in Zoho API Console, then run: npm run get-refresh-token -- <GRANT_CODE> --write
Status line: <i>Unparseable due date</i>	CRM deal contains non-ISO date string (e.g. '10/03/2026' or 'invalid').	Review sample dates in warning message. Locate record in Zoho CRM and update date field to standard calendar date format.
Browser shows stale UI after deploy	Browser cached old static JS files.	Automatic cache-buster stamps URLs with ?v=BUILD_ID. Perform hard refresh (Ctrl+F5) if needed.

End of Documentation -- Zoho Payment Reconciliation & Collections Intelligence Platform